

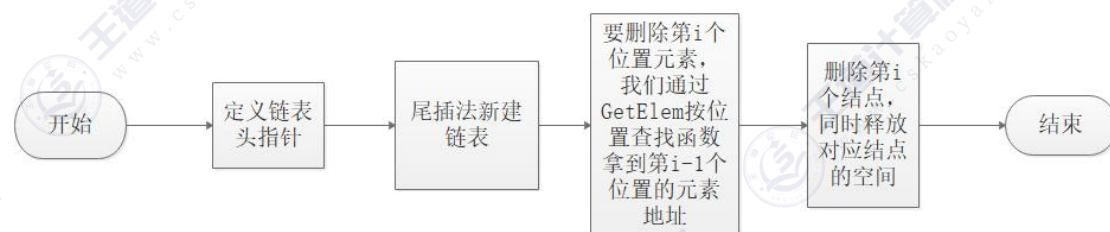
本小节内容

单链表的删除操作实战

单链表的删除操作实战

在第 10.5 小节我们已经讲解了链表的新增，删除，查找的原理。

下面我们绘制了单链表的删除操作实战流程图，不会自己独立写出代码的同学，可以看看视频，看看老师写的代码，接着自己先画一个流程图，再根据自己画的流程图来写代码，流程图的详细情况根据个人来定制，并不一定要和老师的完全一致。



本小节是实战删除链表的第 i 个结点，下面直接上代码（代码包含之前链表操作的代码，只看删除部分即可）：

```
#include <stdio.h>
#include <stdlib.h>

typedef int ElemType;
typedef struct LNode{
    ElemType data;
    struct LNode *next;//指向下一个结点
}LNode,*LinkList;
//头插法新建链表
LinkList list_head_insert(LinkList &L)
{
    LNode *s;int x;
    L=(LinkList)malloc(sizeof(LNode));//带头结点的链表
    L->next=NULL;//L->data 里边没放东西
    scanf("%d",&x);//从标准输入读取数据
    //3 4 5 6 7 9999
    while(x!=9999){
        s=(LNode*)malloc(sizeof(LNode));//申请一个新空间给 s，强制类型转换
        s->data=x;//把读取到的值，给新空间中的 data 成员
        s->next=L->next;//让新结点的 next 指针指向链表的第一个元素（第一个放我们数据的元素）
        L->next=s;//让 s 作为第一个元素
        scanf("%d",&x);//读取标准输入
```

```
    }  
    return L;  
}  
//尾插法新建链表  
LinkedList list_tail_insert(LinkedList &L)  
{  
    int x;  
    L=(LinkedList)malloc(sizeof(LNode));//带头节点的链表  
    LNode* s, * r = L;//LinkedList s,r=L;也可以, r 代表链表表尾结点, 指向链表尾部  
    //3 4 5 6 7 9999  
    scanf("%d",&x);  
    while(x!=9999){  
        s=(LNode*)malloc(sizeof(LNode));  
        s->data=x;  
        r->next=s;//让尾部结点指向新结点  
        r=s;//r 指向新的表尾结点  
        scanf("%d",&x);  
    }  
    r->next=NULL;//尾结点的 next 指针赋值为 NULL  
    return L;  
}
```

//按序号查找结点值

LNode *GetElem(LinkedList L,int i)

```
{  
    int j=1;  
    LNode *p=L->next;  
    if(i==0)  
        return L;  
    if(i<1)  
        return NULL;  
    while(p&&j<i)  
    {  
        p=p->next;  
        j++;  
    }  
    return p;  
}
```

//按值查找

LNode *LocateElem(LinkedList L,ElemType e)

```
{  
    LNode *p=L->next;  
    while(p!=NULL&&p->data!=e)  
        p=p->next;  
    return p;  
}
```

```
}  
//新结点插入第 i 个位置  
bool ListFrontInsert(LinkList L,int i,ElemType e)  
{  
    LinkList p=GetElem(L,i-1);  
    if(NULL==p)  
    {  
        return false;  
    }  
    LinkList s=(LNode*)malloc(sizeof(LNode));//为新插入的结点申请空间  
    s->data=e;  
    s->next=p->next;  
    p->next=s;  
    return true;  
}  
//删除第 i 个结点  
bool ListDelete(LinkList L,int i)  
{  
    LinkList p=GetElem(L,i-1);  
    if(NULL==p)  
    {  
        return false;  
    }  
    LinkList q;//用来存储要删除的结点地址  
    q=p->next;  
    p->next=q->next;//断链  
    free(q);//释放对应结点的空间  
    return true;  
}  
//打印链表中每个结点的值  
void print_list(LinkList L)  
{  
    L=L->next;  
    while(L!=NULL)//NULL 是为了代表一张空的藏宝图  
    {  
        printf("%3d",L->data);//打印当前结点数据  
        L=L->next;//指向下一个结点  
    }  
    printf("\n");  
}  
//《王道 C 督学营》课程  
  
int main()  
{
```

```
LinkList L;//链表头，是结构体指针类型
//LinkList search;//用来存储拿到的某一个节点
//list_head_insert(L);//输入数据可以为 3 4 5 6 7 9999,头插法新建链表
list_tail_insert(L);//输入数据可以为 3 4 5 6 7 9999
print_list(L);//链表打印
//search=GetElem(L,2);
//if(search!=NULL)
//{
//    printf("按序号查找成功\n");
//    printf("%d\n",search->data);
//}
//search=LocateElem(L,6);//按值查询
//if(search!=NULL)
//{
//    printf("按值查找成功\n");
//    printf("%d\n",search->data);
//}
//ListFrontInsert(L,2,99);//新结点插入第 i 个位置
//print_list(L);
ListDelete(L,4);//删除第 4 个结点
print_list(L);
return 0;
}
```